

RAG Deep Dive

10-page practical study guide covering retrieval, generation, RAG variants, guardrails, LangSmith tracing, evaluation, and production design.

1. RAG Fundamentals

Retrieval-Augmented Generation (RAG) is an application architecture in which a language model receives relevant external information at generation time. Instead of relying only on knowledge stored in model parameters, the application retrieves information from a knowledge source and places that information into the model context.

A basic RAG pipeline has two broad phases: retrieval and generation. A typical flow is user question → query representation → retrieval → relevant document chunks → prompt construction → language model → answer.

RAG is useful for private enterprise information, frequently changing documents, and applications where answers should be tied to source material. It can also make answers more auditable because the application can preserve the passages that supported a response.

RAG is a system, not just a model feature. Parsing, chunking, embeddings, retrieval, prompt design, generation, security, and evaluation all affect the final result.

2. Document Ingestion and Loading

RAG begins with source data such as PDFs, Word documents, web pages, support tickets, databases, product manuals, and internal knowledge bases. A loader converts a source into a common document representation containing text plus metadata.

For PDFs, an application may extract page text while retaining filename and page number. Metadata can later support filtering, source display, access control, and debugging. Extraction quality matters for scanned PDFs, tables, multi-column layouts, headers, footers, and image-heavy documents.

A robust ingestion process can include file-type validation, size limits, duplicate detection, text cleanup, encoding normalization, metadata enrichment, and error handling. In production, ingestion is normally separated from question answering so embeddings are not recreated for every query.

Example: an employee handbook can be loaded into page-aware documents. A question about parental leave can later retrieve the relevant policy page and its metadata.

3. Chunking and Text Splitting

Large documents are divided into smaller units called chunks. A common starting point is a recursive character text splitter with a target chunk size and overlap.

Chunk size is a retrieval design choice. Very small chunks can lose context; very large chunks can contain unrelated material and reduce retrieval precision. Overlap helps preserve information that crosses chunk boundaries.

Structure-aware chunking can outperform blind character cutting. Technical documentation may be split around headings, paragraphs, lists, or code blocks. Contracts may be split around clauses and sections. The goal is to create chunks that can independently provide useful evidence.

Example: a 300-page manual may become thousands of chunks. A question about installation should retrieve installation sections rather than unrelated troubleshooting material.

4. Embeddings and Vector Databases

An embedding model converts text into a numerical vector intended to represent semantic characteristics. During ingestion, each chunk is embedded and stored in a vector database together with its text and metadata.

At query time, the question is embedded too. The vector database compares the query vector with stored chunk vectors and returns nearby candidates according to a similarity measure. This supports semantic matching even when the question and source use different wording.

Vector stores such as Chroma or FAISS can support this pattern. The appropriate production choice depends on persistence, scale, filtering, multi-user requirements, operations, and deployment architecture.

Example: the question 'How can I get my money back?' may retrieve a section titled 'Refund Procedure' even when the exact wording is different.

5. Retrieval and Choosing k

A retriever converts a query into relevant documents or chunks. In LangChain, a vector store can be exposed as a retriever. A setting such as `search_kwargs={'k': 4}` means four candidate chunks are returned; it does not mean four PDF pages.

The correct k depends on the question and chunking strategy rather than simply document length. A small k can miss evidence; a large k can introduce irrelevant context, increase latency and token usage, and reduce focus.

Complex applications can use multi-stage retrieval. A first retriever can return a larger candidate set, after which a reranker selects the strongest passages. Metadata filters can restrict search by department, customer, date, document type, or permissions.

A useful retrieval test asks whether the retrieved context contains the evidence needed to answer the question. This separates retrieval quality from generation quality.

6. Prompting and Generation

After retrieval, the application constructs a prompt containing the user's question and selected context. A strong RAG prompt treats retrieved documents as reference material and tells the model what to do when the answer is absent.

A typical instruction is to answer using the supplied context and say that the answer was not found when the context lacks it. This establishes intended behavior and can reduce unsupported answers.

The model can be integrated through a LangChain chat model wrapper. Model configuration should come from application configuration or environment variables. API keys should never be hard-coded or committed to Git.

Generation should ideally preserve source references so users can verify important claims.

7. RAG Variants and When to Use Them

Basic or naive RAG is appropriate for straightforward document question answering: retrieve semantically similar chunks and pass them to the model.

Hybrid RAG combines semantic vector search with lexical retrieval such as BM25. It is useful when exact terms matter, including product codes, policy IDs, names, legal clauses, error codes, and technical identifiers. A reranker can reorder combined candidates.

Multi-query or fusion approaches use multiple search formulations and combine results. Parent-child retrieval searches smaller child chunks but can return a larger parent section for generation.

Corrective RAG evaluates whether retrieved results appear useful and can take corrective retrieval actions. Self-RAG adds self-evaluation around retrieval and generation. Conversational RAG is useful when follow-up questions depend on conversation context. Agentic RAG is useful when the system must choose among multiple tools or knowledge sources.

8. Guardrails and Security

Production RAG needs guardrails around the model and retrieval pipeline. Input guardrails can validate question length, file type, file size, and application scope. Document guardrails can reject malformed files and enforce ingestion limits.

Prompt injection is a major RAG concern because retrieved text can contain instructions that should not be followed. Retrieved documents should be treated as untrusted data, not higher-priority instructions. System instructions should clearly separate trusted application instructions from retrieved content.

Retrieval guardrails can detect weak evidence. If relevance is below a threshold, the system can say the information was not found instead of generating a confident unsupported answer. Output checks can look for unsupported claims and application-specific policy violations.

Access control is critical: a vector database should not return documents a user is not authorized to see.

9. LangSmith Tracing and Evaluation

Observability makes RAG debuggable. LangSmith tracing can capture execution across retrieval, prompt construction, model calls, and other LangChain components. A trace can show questions, retrieved documents, prompts, responses, latency, and errors depending on configuration.

Evaluation should test multiple dimensions. Retrieval relevance asks whether useful evidence was retrieved. Answer relevance asks whether the response addresses the question. Groundedness asks whether the answer is supported by retrieved context. Correctness compares the answer with a trusted reference when available.

A practical evaluation dataset contains representative questions and, where possible, reference answers or expected evidence. Run the same dataset against different chunk sizes, k values, retrieval strategies, prompts, or models.

Example: compare Basic RAG with Hybrid plus Rerank using retrieval relevance, groundedness, latency, and cost. The best design is the one that meets the application's quality and operational requirements.

10. Production Design and Question Bank

A production-oriented PDF RAG system should separate ingestion from query-time retrieval, persist its vector index, validate uploaded files, handle errors, protect secrets, apply access control, and provide source information with answers. Streamlit can provide the UI while LangChain manages the retrieval and model pipeline.

A sensible progression is PDF upload → loading → chunking → embeddings → vector store → retriever → OpenAI model → answer. Then add metadata, persistence, source citations, hybrid retrieval, reranking, guardrails, tracing, and evaluation.

Core questions: What is RAG? Why is RAG needed? Explain the complete RAG pipeline. Why chunk documents? How do chunk size and overlap affect retrieval? What are embeddings? Why use a vector database? What does k mean? How do you choose k for a long PDF? What is hybrid search? Why use BM25? What is reranking? When would you use multi-query RAG? What is corrective RAG? How can prompt injection affect RAG? How do you evaluate retrieval quality? What is groundedness? How does LangSmith help debug RAG?

Scenario questions: A 1,000-page manual returns irrelevant chunks; what would you change? Users search for exact policy IDs; should you use vector search alone? The answer sounds plausible but is not supported by the PDF; which evaluation or guardrail should detect this? A user asks a follow-up question referring to the previous answer; which RAG pattern is appropriate? Two departments have private documents; how should retrieval enforce access control?